Fenton production solver — final verification

Engineering conclusion

The script computes accurately for most configurations in the benchmark matrix. Nineteen of the 21 physical cases satisfy all acceptance requirements on both NumPy and Numba: **38 accepted runs out of 42**. These include the default H=3 m, T=9 s, d=5 m cases, small period waves, opposing current, and both current definitions. The same two steep-wave configurations on each backend are explicitly rejected because their spatial resolution does not meet the output accuracy budgets.

The mathematical formulas, independent high-precision calculations, runtime acceptance checks and Lean statements support this conclusion for the tested configurations. The benchmark is not a statistical sample of all possible inputs, so its approximately 90% acceptance rate must not be interpreted as a universal success probability.

The scope of formal certification is precise: Lean verifies the stated mathematical identities, enclosure rules and recorded inequalities. The entire Python execution and the continuous water-wave boundary-value problem are not formally certified. This scope limitation does not mean that the successful engineering calculations are inaccurate.

Verification results

Evidence	Result
Physical cases	21 cases on each of NumPy and Numba.
Accepted results	38 runs covering 19 physical cases.
Accuracy-limited results	Four runs covering two physical cases; no accepted output is displayed for them.
Finite equations enclosed	4,348 equations across the 42 stored states.
Largest residual enclosure bound	Approximately `2.177512e-15`; acceptance requires no more than `1e-10`.
Selected output enclosures	Nine quantities per state; 378 output enclosures and error budgets.
Independent enclosure cross-check	All residual and selected-output enclosures agree with 80-digit evaluations. A `1e-70` comparison allowance covers rounding of the independent reference at degenerate endpoints.
Formula Lean module	62 theorems.
Acceptance Lean module	13 theorems.
Finite-system Lean module	29 theorems.
Numerical-observation Lean module	314 theorems.
Finite-certificate Lean module	420 theorems.
Total Lean statements	**838 theorems**, all compiled.
Interval arithmetic tests	250 signed interval families, checked using exact rational arithmetic at endpoints and interior points.
Deliberate perturbations	Altered Fourier state and altered power output rejected; restored state accepted.

Evidence	Result
Report tests	Magnitude labels, partial-result suppression, full rejection and power-perturbation checks pass.
Built-in tests	22 acceptance/rejection checks pass across both backends.

Additional independent tests cover two complete analytic Jacobians, point kinematics, surface-node reconstruction, integral quadrature refinement, input and stagnation guards, and three deliberately difficult nonlinear targets. All defined test gates pass. The full evidence is in `results/`.

Finite Fourier-system verification

After solving, the program independently evaluates all `2N+10` finite-system equations using **50-digit outward-rounded Decimal intervals**. Input binary64 values are interpreted exactly. The check reconstructs the nondimensional height, period and current quantities from the physical inputs, independently of the solver's rounded intermediate values.

The equations comprise the eight global constraints, N+1 surface-streamline equations and N+1 Bernoulli equations. Trigonometric factors use a separate Taylor evaluator at the mathematical collocation phases. The interval is

`3.14159265358979323846` `3.14159265358979323847`

Both inequalities are proved in Lean using Mathlib. Angles are reduced to `[- , ]`; the sine and cosine polynomials contain 32 terms, with analytical remainder bounds `|x|^65/65!` and `|x|^64/64!`. Arithmetic endpoints use directed rounding. Exponentials and square roots use correctly rounded Decimal operations expanded to neighbouring representable values.

The resulting intervals enclose the equation values at the stored finite Fourier state, subject to the stated implementation trust boundary. Every recorded interval endpoint is checked in Lean against the residual tolerance. This verifies a small residual at that state; it does not establish that a mathematically exact root exists nearby or that it is unique.

Selected outputs

The interval checker evaluates the following finite-formula quantities independently:

Quantity	Definition checked
Wavelength `L`	`2 /k`, with `k=z /d`.
Celerity `c`	`z  sqrt(g/k)`.
Impulse	The finite-state flux/current identity with dimensional scaling.
Kinetic energy	The finite-state energy/flux identity with dimensional scaling.
Potential energy	The squared surface coefficients, including the terminal half-amplitude weight.
Total energy density	Kinetic plus potential energy.

Quantity	Definition checked
Radiation stress `Sxx`	The invariant expression with the selected current convention.
Power	The invariant energy-flux expression with current.
Mean-square orbital bed velocity	The Fourier sum with hyperbolic attenuation at the bed.

These enclosures concern evaluation of the formulas at the stored coefficients. They do not include uncertainty in physical inputs, error between the numerical state and an exact root, or truncation error relative to the continuous wave.

Each reported value is compared with its entire enclosure. Its worst-case distance must fit the dimensional budget

```
1e-7 + 1e-8 × |reference|
```

The calculation uses the minimum possible reference magnitude over the interval, yielding a conservative budget. Lean checks the exact endpoint, error and budget inequalities after scaling them to integers. The generic real-number implications and scaling rules are proved in `FiniteSystem.lean`.

Spatial-resolution and output checks

Finite-system verification is supplemented by a scan of at least 2,048 interpolated surface phases, spectral bed-velocity comparisons, and two resolutions of pressure/velocity quadrature. The maximum surface residual must not exceed `1e-6`. Impulse, kinetic energy, potential energy, radiation stress and power must satisfy the dimensional budget above for both invariant-versus-integral agreement and quadrature refinement.

These checks address consistency and resolution beyond the collocation nodes. They are numerical diagnostics rather than rigorous continuous-phase or quadrature remainder bounds. Their tolerances are explicit acceptance choices, not universal tolerances prescribed by Fenton.

The two configurations requiring more resolution are:

H (m)	T (s)	d (m)	Eulerian U (m/s)	Order used	Surface residual, approximately
2	3	5	0	32	`1.18e-5`
3	4	5	1	40	`7.82e-5`

Their finite collocation equations have small residuals, but the interpolated wave and integral comparisons exceed the spatial-resolution budgets. The program therefore returns `resolution_failure`. This is a deliberate accuracy decision, not evidence that all short-period waves fail or that these physical waves cannot exist.

Runtime status and interpretation

- `converged`: a result has passed the solver and all runtime acceptance checks.
- `accuracy_verified`: the numerical output and resolution checks passed.
- `finite_system_verified`: the finite residual and selected-output interval checks passed.
- `finite_system_certificate`: all decimal interval endpoints, residual bounds, reported values and output budgets for the stored state.
- `accuracy_metrics`: numerical comparison ratios and the surface residual.
- `solution_status` and `last_error`: accepted state or an explicit reason for rejection.

`finite_system_verified` alone is not sufficient for an accepted result: spatial-resolution checks must also pass. The report displays the finite-system residual bound for accepted cases. The certificate is also accessible through the solver API without starting the GUI.

Horizontal velocity magnitudes and surface acceleration are phase-refined estimates. Signed crest velocities are separate fields. Bed shear and excursion remain engineering diagnostics. `Cg` is reported as energy-flux velocity `F/E`, and Miche's criterion is a breaking diagnostic, not a stability proof.

The separate moving-frame momentum-flux output uses a single-phase trapezoidal calculation. Its independent test budget is `1e-6 + 1e-5*abs(reference)`; it is not one of the nine interval-enclosed outputs.

Formal proof boundary

The Lean kernel checks every delivered theorem. The 420 finite-certificate theorems use kernel-evaluated exact integer decisions and Lean's standard `propext`. Other modules use only the standard `propext`, `Classical.choice` and `Quot.sound` where required. No `sorry`, `admit`, `native_decide` or project-defined axioms are used.

The correspondence between the Python source and the real-number model is reviewed and tested. Decimal's implementation, the Python interval evaluator, its Taylor remainder implementation, NumPy/Numba execution and the GUI are not extracted from Lean or proved as complete programs. In particular, the package does not contain a validated root-existence/uniqueness certificate, a formal general DCT inversion proof, or a continuous free-boundary truncation bound.

These boundaries define what the evidence establishes. They do not negate the demonstrated agreement of the accepted cases with independent mathematical evaluations.

Sources and reproduction

Source hashes, mathematical references and the function map are included. The exact production Python file and the mathematical README are preserved under `inputs/`. Use the package README to reproduce the numerical and Lean checks.

- Fenton (1988), The numerical solution of steady water wave problems.
- Fenton (1990), Nonlinear wave theories, section 7 for integral quantities and current conventions.
- Fenton, Instructions for FOURIER, CNOIDAL and STOKES, appendices A and B.
- Python Decimal documentation, directed rounding and correctly rounded elementary operations.

- Mathlib `Analysis/Real/Pi/Bounds.lean`, pinned by the included Lake configuration.

The verification environment used Python 3.12.14 and Lean 4.34.0. Lean modules were compiled directly against the pinned Mathlib cache. An executable-path lookup shim was needed by the execution environment to locate the compiler installation; it did not modify the Lean kernel or proof rules. An ordinary Lean installation does not require it.